

Adding Semantic Structure to Unstructured PDFs via the Unix Command Line

Andrew J. Hayes

May 16, 2026

Introduction and Use Case

This paper demonstrates the concept and simple implementation for a workflow that an individual scholar may use to enhance the utility of a private research corpus of ancient or medieval sources in pdf format by adding a semantically structured document outline to them. Traditional papers and books are usually divided into sections, each with a header, and (in the case of books) often furnished with some kind of table of contents to facilitate rapid navigation to a particular point of interest in that structure. The pdf standard, which intends to codify a digital representation of paper documents that would be portable across different operating systems,¹ has long supported a digital implementation of this feature: a navigable table of contents, usually in a sidebar. But, despite the fact that papers, books, and theses are now commonly accessed in pdf form, they often lack such an outline. This paper shows how to automate the process of adding one using Unix command line tools.

This workflow will only be useful if the document in question is composed of searchable digital text, either because it was born digital, or because optical character recognition (OCR) has subsequently added a text layer to each page. It is not suited for very short texts or texts that inherently lack a discernible structure or possess it only minimally, such as scans of medieval manuscripts. The most usual application will

¹ *ISO 32000-2 Portable Document Format – Part 2: PDF 2.0*, 2020, viii (§0.1); Jonas Gamalielsson and Björn Lundell, *Experiences from Implementing PDF in Open Source: Challenges and Opportunities for Standardisation Processes*, vol. 2013 8th International Conference on Standardization and Innovation in Information Technology (SIIT) (IEEE, 2013).

be to critical editions of primary sources or to the translation volumes that accompany them. A logical secondary application would be to longer articles with many sections.

This presentation aims to show what is possible using open source Python tooling and to provide a simple implementation of command line scripts that the reader may use and adapt for his or her own specific needs. Along the same lines, the paper includes a few observations about what was needed to implement the scripts in Python and POSIX shell, with the hope that this will help others learn enough to adapt the scripts to their purposes and avoid pitfalls. This paper and the accompanying source code will be released under an open source license. Code may be updated in the future to account for changes in the Python ecosystem, or the Docling library on which the code depends. The workflow demonstrated here is compatible with the one demonstrated in the author’s paper: “Efficient Vocabulary Discovery in Late Antique Texts” given at the 2025 Annual Meeting of the North American Patristics Society, and [publicly available here](#).²

² **Note on AI Use:** Generative AI was not used to research the ideas and techniques contained in, nor produce the text of, this paper or its associated source code.

Brief Conceptual Background to the Tooling

Python is a general purpose interpreted programming language widely used in scientific research and in academia more generally. It enjoys a rich standard library, and can be used (among many other applications) to write composable command line scripts, following the Unix software tools approach.³ According to this approach tools should ideally perform a single task well, but be designed to receive and pass along input and output in a standard way, to enable composition. One achieves such composition by in-

³ Brian W. Kernighan and P. J. Plauger, *Software Tools* (Reading, Massachusetts: Addison-Wesley Professional, 1976), 1–6.

terleaving calls to specific commands or scripts with the pipe character (`|`).

The output of one program becomes the input of the next in the pipeline until the final desired result is output.⁴ The workflow demonstrated in this paper uses the Python standard library in conjunction with Docling to script a pipeline that takes a pdf lacking a digital Document outline, creates such an outline by parsing the document's section headers, adjusts it, and applies it to a new copy of the pdf.

⁴ Richard Blum and Christine Bresnahan, *Linux Command Line and Shell Scripting Bible* (John Wiley & Sons, 2015), 279–84.

Docling is relatively recent open source python library and command line tool, developed by IBM for parsing pdfs and other human-readable documents⁵ so that their data can be more easily ingested by another application, such as a large language model and thus augment it with domain-specific knowledge. This typical use-case is known as RAG (Retrieval Augmented Generation).⁶ This is a broad domain, and the Docling library is powerful and feature-rich. Nevertheless, this workflow uses only a small subset of its abilities for a comparatively simple task: extracting the names and locations of section headings in a structured form that can be manipulated programmatically.

⁵ “Docling” (<https://docling-project.github.io/docling/>, n.d.).

⁶ “Retrieval-Augmented Generation (RAG) Vs. Fine-Tuning,” The official Red Hat blog (<https://www.redhat.com/en/topics/ai/rag-vs-fine-tuning>, 2026).

Pdfcpu is an open source library and command line tool written in Go which is able to perform a broad range of pdf manipulations.⁷ In this workflow it is used to apply the tree structure of the document outline to the pdf, as the final stage in the workflow.

⁷ “Pdfcpu: A PDF Processor Written in Go” (<https://pdfcpu.io>, n.d.).

It is important to note that the terminology for the pdf document outline varies. The current PDF ISO standard describes it this way:

The outline consists of a tree-structured hierarchy of outline items (sometimes called bookmarks), which serve as a visual table of contents to display the document's structure to the user.⁸

⁸ *ISO 32000-2 Portable Document Format – Part 2*, sec. 12.3.3.

The wording of the standard suggests that the canonical name for the items in the hierarchy is “outline items.” Adobe Acrobat’s user interface at the time of writing refers to the items in the structure as bookmarks. Pdfcpu’s documentation also employs this terminology. Note that bookmarks need not form part of a tree. They can take the form of navigation targets in a flat list. In this paper such bookmarks or document outlines and their outline items will also be referred to generically as tables of contents.

Workflow

Cautionary Note:

When working with any important file, such as laboriously obtained scans of physical originals, it is good practice always to edit copies rather than overwriting the original files.

The workflow consists of three phases: generating, updating, and applying the table of contents. Below, I offer an outline of these three phases, followed by some practical considerations for real-world use.

Phase One: Generate the Table of Contents

In order to add a table of contents to a pdf, one first has to derive the information from the file and represent it in a structured way. To do this, we use Docling’s ability to process a pdf and export a stream of JSON⁹ (which can also, if desired, be saved to a file). This JSON contains Docling’s “understanding” of the whole document, but to create a document outline we need to extract only the information about section headings from the JSON Docling produces.

⁹ “Introducing JSON” (<https://www.json.org/json-en.html>, n.d.); *Standard ECMA 404: The JSON Data Interchange Syntax*, 2017.

At least two additional practical considerations are relevant. First, calls to Docling are time consuming and computationally expensive. For this reason, the script caches the result of a previous run. A full re-parse of the document can be forced by invoking the `tcgen` script with the `-B` flag. Second, since medievalists and humanists more generally often have to work with texts in languages such as Latin or Greek, it is important to output the json text in utf-8 encoding. In the script, this is done by ensuring that calls to Python's `json.dumps` (dump string) function are passed the `ensure_ascii=False` parameter. Users of the script must, of course, have a unicode-capable terminal emulator and the appropriate fonts installed.

In our example, we will use a born-digital pdf critical edition of the letters of the *Corpus Dionysiacum*.¹⁰ Note that, due to copyright, it cannot be distributed in the repository along with this paper. Readers are encouraged to supply their own pdf file(s) for experimentation. Despite being purchased as a natively digital pdf, it lacks any document outline. It contains the usual sorts of headers that a printed critical edition might contain: an introductory section for *sigla* used in the edition, followed by the critical text of each letter, whose titles, like the letters themselves, are printed in Greek. To generate a table of contents, we simply invoke the script with the filename as an argument:

¹⁰ Pseudo-Dionysius Areopagita, *Corpus Dionysiacum, II*, ed. Günther Heil and Adolf Martin Ritter, vol. 36 (Berlin: Walther de Gruyter, 1991).

```
./tcgen ./sources/Pseudo-Dionysius_Areopagita_1991_416.pdf
```

This outputs a json string containing an array, each element of which is a json object containing a title key and a page key, indicating the title of the section heading and the absolute page number of the pdf on which it

occurs.

The resulting json representation of the table of contents contains some errors, though they are not easy to notice because the output is visually compressed. We will show a way to address that in the next section.

Phase Two: Update the Table of Contents

The output of `tcgen` is imperfect. It contains some (semantically) duplicate nodes, because Docling treated some headers as two separate headers on the same page. It also contains some basic labelling mistakes. How can we more easily spot them? `tcedit` offers a way.

The script `tcedit` receives the json stream from the previous command (here `tcgen`). When given no options, it outputs an enumerated list of bookmark nodes so that the user can inspect them and decide what changes might be necessary. This intermediate representation is intended for the user and not for passing on to another command in the pipeline. In this case, if we invoke it with the same filename as before:

```
./tcedit ./sources/Pseudo-Dionysius_Areopagita_1991_416.pdf
```

we will see the enumerated list of bookmark nodes. This reveals the duplicated nodes, as well as nodes that mistakenly treated Greek text like Roman script.

It can be invoked with the `-d` (or in long form `--delete`) flag, along with the index numbers of the nodes to delete. For example, to delete the seventh and the thirteenth bookmark, one could invoke it as follows:

```
./tcgen ./sources/Pseudo-Dionysius_Areopagita_1991_416.pdf | \
./tcedit -d 7 13
```

It will output the json string to standard output with the requested nodes deleted.

Using the `-u` (long form `--update`) flag, it is possible to make changes to the text of a given node using a `KEY=VALUE` syntax in which the `VALUE` is itself a tuple consisting of the portion of the node that needs editing. For example:

```
./tcgen sources/Pseudo-Dionysius_Areopagita_1991_416.pdf | \
./tcedit -u 6="title,y TQI AYTQI" \
8="title,δ TQI AYTQI" \
14="title,θ TITQI ?IEPAPXHI" \
-d 7 13
```

This pipeline updates the title portions of nodes 6, 8, and 14. Both the `-u` and `-d` flags may be used together with the same command. The order in which they are given does not matter. `tcedit` will always perform update operations before the delete operations in a single invocation. Once we are satisfied that the edits are correct, the `-f` (long form `--final`) flag can be added to output it in the form needed for `Pdftcpu` to apply it to the finished pdf.

Phase Three: Apply the Table of Contents to the PDF file

The final stage of the workflow is to apply the output of the `tcedit` script to a copy of the original pdf file and output it. A simple shell script, `tcApply.sh` has been provided for this purpose:

```

./tcgen sources/Pseudo-Dionysius_Areopagita_1991_416.pdf | \
./tcedit -u 6="title,γ TQI AYTQI" \
8="title,δ TQI AYTQI" \
14="title,θ TITQI ?IEPAPXHI" \
-d 7 13 -f | \
./tcApply.sh sources/Pseudo-Dionysius_Areopagita_1991_416.pdf
    
```

Note the use of the `-f` flag to finish outputting the result of `tcedit`, which is then piped into the `tcApply.sh` script, which produces a file called simply `output.pdf`. This file contains the navigable table of contents. It is now ready to use. Ordinarily, one would then meaningfully rename the file and store it in a personal digital library of texts.

Considerations in Real-World Use

On the Unix command line, it is often useful to compose a pipeline with repeated invocations of the same command but with different flags or arguments. Doing this makes it easy to experiment and build up incrementally the desired result. Because the source is never modified directly, one can always repeat new variations of the pipeline until the desired result is achieved, which can then be written to a file. `tcedit` largely conforms to this usage pattern. It can be applied multiple times in the pipeline, breaking up a complex invocation into simple parts. It can always be added to the chain without a flag to inspect the output at that point in the transformation process.

It may also happen that one needs to stop work in the midst of an exploratory session in which one has not completed all desired trans-

formations. In this case, you can redirect the bookmarks object to a file (equivalent to issuing a “Save” command via a GUI). It is possible to reload this file later to resume the transformation process. Here is what that would look like:

```
./tcgen sources/Pseudo-Dionysius_Areopagita_1991_416.pdf |\
./tcedit -u 6="title,γ ΤΩΙ ΑΥΤΩΙ" \
8="title,δ ΤΩΙ ΑΥΤΩΙ" \
14="title,θ ΤΙΤΩΙ ?ΙΕΡΑΡΧΗΙ" \
-d 7 13 > unfinished.json
```

This would save the bookmark list to the file `unfinished.json`. It could easily be picked up again using the standard Unix tool `cat`:

```
cat unfinished.json | ./tcedit -u \
13="title,ι 'ΙΩΑΝΝΗΙ ΘΕΟΛΟΓΩΙ, ΑΠΟΣΤΟΛΩΙ ΚΑΙ ΕΥΑΓΓΕΛΙΣΤΗΙ ΠΕΡΙΟΡΙΣΘΕΝΤΙ ΚΑΤΑ ΠΑΤΜΟΝ ΤΗΝ ΝΗΣΟΝ" | ./tcedit
```

Here, the contents of the `unfinished.json` file are piped back into the `tcedit` script to complete one more update. The last node in the bookmark list does not begin with a lowercase Greek letter. All the other section headers begin with their appropriate Greek letter in sequence. But with this final transformation accomplished, the last section header matches the format of all the rest.

Docling’s ability to output pdf structure as json is a significant advantage because json is so widely used and widely supported. In fact, for more advanced transformation, it is possible to dispense with the `tcedit` script and use `jq`, a robust command line tool for filtering json.¹¹ Just like any other Unix filter, it can be added to the pipeline to achieve whatever transformation is desired. `jq` is out of scope for this tutorial, but it shows

¹¹ “/Jq” (<https://jqlang.org>, n.d.).

what is possible with composable command line tools.

A digital humanist could easily modify and extend these scripts. One could, for example, create a version of these tools that would output just the text of each heading with or without page numbers. This could be redirected to another file and turned into the skeleton of a handout for a talk. This output could in turn be modified or piped into pandoc to produce the final handout. Docling is not limited to ingesting pdfs. It can also ingest $\LaTeX$ files and MS Word files.

Limitations and Summary

The workflow offered here makes the most sense as a tool for an individual scholar or a small team with limited budget. Alternatives exist. One could use a variety of commercially available GUI software tools to edit the document outline of an existing pdf. The most obvious one is Adobe Acrobat, distributed by the company that developed the PDF standard originally. PDFExpert is a powerful (albeit Mac only) alternative. Xodo distributes a free pdf reader that can edit document outlines. Other commercially available software, including web-based tools may have similar or even more advanced capabilities. But of course, commercial solutions cost money and cannot be freely adapted like open source software can. Perhaps more importantly for efficient processing of large personal libraries, they are more complex to automate or cannot be automated at all. The primary advantage that command line tools provide is that Docling can generate a reasonable approximation whose results can be processed in a batch or in an automated fashion. Creating a document outline for a large critical edition by hand would be laborious and error

prone.

The scripts offered in this tutorial are proof-of-concept. They have immediate utility, but also a few key limitations that one might want to improve upon in a version adapted for personal use. Most notably, `tcedit` lacks the ability to perform more complex transformations beyond simple node deletion and editing. It cannot nest them as sub-headings. This limitation is also reflected in `Docling`, which does not seem to be able to reliably interpret the section header hierarchy. In the interest of speed, the `tcgen` script does not perform OCR directly, though this is a capability that `Docling` offers, and it could be added by adapting the `tcgen` script and installing OCR dependencies on the local system.

`Docling` allows the creation of powerful command line tools in Python for extracting the information needed for a pdf document outline. `Pdfcpu` makes it easy to apply this document outline to a file. The result is a more useful digital text. The Unix Software Tools philosophy makes this utility possible by creating an environment in which commands can be iteratively composed, experimented with, and automated. And such tools are available for free with few restrictions on their use.

Bibliography

Blum, Richard, and Christine Bresnahan. *Linux Command Line and Shell*

Scripting Bible. John Wiley & Sons, 2015.

“Docling.” <https://docling-project.github.io/docling/>, n.d.

Gamalielsson, Jonas, and Björn Lundell. *Experiences from Implementing PDF in Open Source: Challenges and Opportunities for Standardisation Processes*. Vol. 2013 8th International Conference on Standardization and Innovation in Information Technology (SIIT). IEEE, 2013.

“Introducing JSON.” <https://www.json.org/json-en.html>, n.d.

ISO 32000-2 *Portable Document Format – Part 2: PDF 2.0*, 2020.

“./Jq.” <https://jqlang.org>, n.d.

Kernighan, Brian W., and P. J. Plauger. *Software Tools*. Reading, Massachusetts: Addison-Wesley Professional, 1976.

“Pdfcpu: A PDF Processor Written in Go.” <https://pdfcpu.io>, n.d.

Pseudo-Dionysius Areopagita. *Corpus Dionysiacum, II*. Edited by Günther Heil and Adolf Martin Ritter. Vol. 36. Berlin: Walther de Gruyter, 1991.

The official Red Hat blog. “Retrieval-Augmented Generation (RAG) Vs. Fine-Tuning.” <https://www.redhat.com/en/topics/ai/rag-vs-fine-tuning>, 2026.

Standard ECMA 404: The JSON Data Interchange Syntax, 2017.